4-bit Array Multiplier from RTL to GDSII

ECE.463 Digital VLSI Design

Mrityunjay Badoni (2026203019) Shreekara Aithala (2026203001)

October 26, 2026

1 Introduction

This report describes the design of a 4-bit × 4-bit unsigned array multiplier and its implementation from register-transfer level (RTL) to a manufacturable GDSII layout. The multiplier forms 16 partial-product bits with AND gates and adds them in three rows of four full adders, each row a 4-bit ripple-carry adder, giving an 8-bit product. Both inputs and the output are registered, so the design contains 16 flip-flops with an asynchronous active-low reset, and the product appears two clock cycles after the inputs are applied.

The design was implemented on the SkyWater SKY130 130 nm process using the high-density standard-cell library `sky130_fd_sc_hd`. Only open-source tools were used, all running in the IIC-OSIC-TOOLS Docker container. Table 1 lists the steps of the flow and the tool used for each.

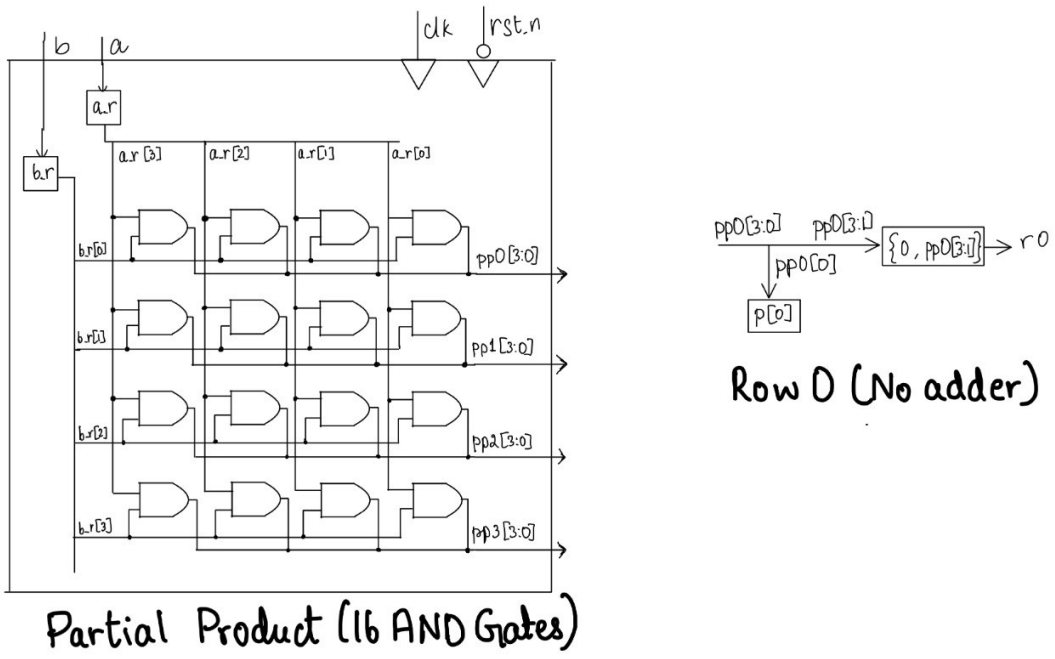
Table 1: Design flow, tools and the report section covering each step.

Step	Tool	Section
RTL simulation and verification	Icarus Verilog 14.0, GTKWave	2
Logic synthesis	Yosys 0.69 (with ABC)	3
Static timing analysis	OpenSTA 3.1.0	4
Place and route, DRC, LVS	LibreLane 3.1.0.dev3 (OpenROAD, Magic, Netgen, KLayout)	5
Layout viewing	KLayout 0.30.12	5

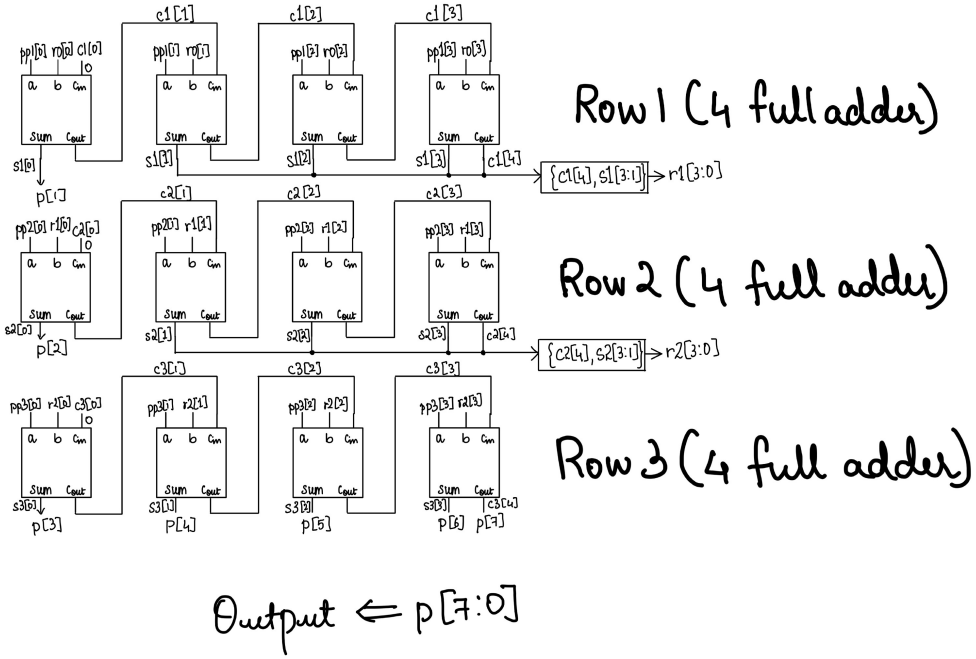
Sections 2 to 5 present the results of Parts A to D of the assignment: design and verification, synthesis, timing analysis and physical design. Section 6 answers the assignment questions and Section 7 concludes. All source files, scripts and generated reports, together with every command that was run, are in the team's Git repository and its `README.md`.

2 Design and Verification (Part A)

The module `full_adder` computes `sum = a ^ b ^ cin` and `cout = (a & b) | (b & cin) | (a & cin)` with two continuous assignments. The top module `mult_array` registers `a` and `b` into `a_r` and `b_r` on each rising clock edge, builds the array from the registered values, and registers the 8-bit result into `p`; all 16 flip-flops are cleared when `rst_n` is low. Row 0 consists only of the AND gates $pp_0[j] = a_r[j] \wedge b_r[0]$ (Figure 1a). Rows 1–3 each add a new partial product to the four bits passed down from the row above, using a 4-bit ripple-carry chain of full adders (Figure 1b). The lowest sum bit of each row gives $p[1]$, $p[2]$ and $p[3]$, and row 3 also gives $p[7{:}4]$. The design uses 16 AND gates, 12 `full_adder` instances and 16 flip-flops, and no `*` operator.



(a) Input registers `a_r` and `b_r`, the 16 AND gates forming the partial products, and row 0.



(b) The three rows of four full adders and the output register `p`.

Figure 1: Block diagram of the multiplier.

The hand-worked example uses Mrityunjay Badoni's roll number (last digit 9): $A = 9 + 5 = 14$ and $B = 7$. Figure 2 shows the four partial products and their sum, $P = 98$ (`01100010`).

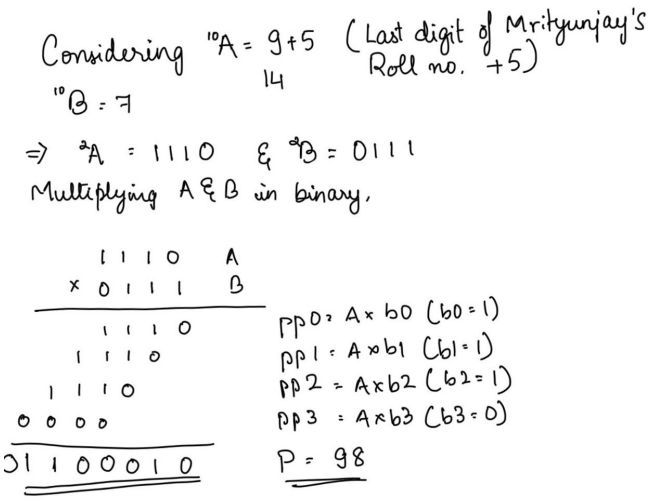


Figure 2: Hand-worked example $14 \times 7 = 98$.

The testbench generates a 10 ns clock, holds `rst_n` low for two cycles, and applies all 256 input pairs on falling clock edges. After the 2-cycle latency, it compares `p` with $a \times b$ and printed `PASS: 256 vectors checked, 0 errors`. With `sum` temporarily changed to `a ^ b`, it printed `FAIL: 256 vectors checked, 95 errors`. The 161 cases that still passed never produce a carry-in of 1 at any full adder (for example when a or b is 0), which shows why the test must be exhaustive.

In Figure 3, $A = 14$ and $B = 7$ are applied at 4640 ns, captured at the rising edge at 4645 ns (edge 1), and $p = 98$ is captured at 4655 ns (edge 2). The latency is therefore **2 clock cycles** (20 ns).

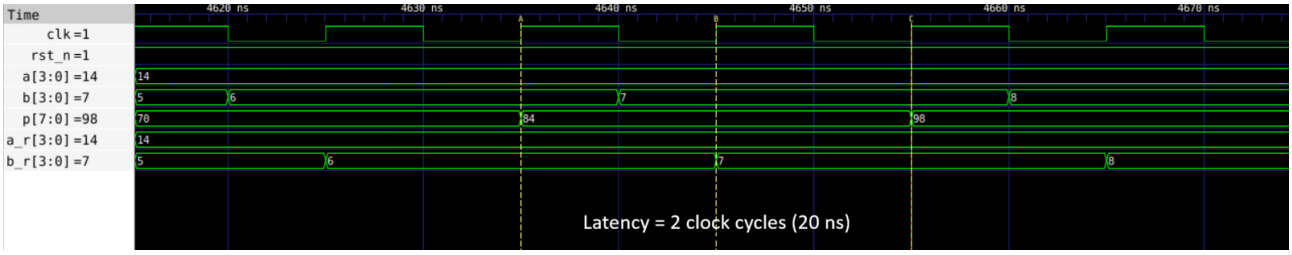


Figure 3: Simulation waveform for $A = 14$, $B = 7$ showing `clk`, `rst_n`, `a`, `b` and `p`, with the 2-cycle latency marked.

3 Logic Synthesis (Part B)

The RTL was synthesized with Yosys using the script `synth/synth.tcl`. After reading both source files, `synth -top mult_array -flatten` performs generic synthesis and dissolves the 12 `full_adder` instances into a single module, so that logic can be optimised across adder boundaries. `dfflibmap` then maps the 16 flip-flops to SKY130 flip-flop cells, and ABC (`abc`) optimises the combinational logic and maps it to SKY130 gates, using the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`. Finally, unused wires are removed, the cell and area report is written, and the gate-level netlist is saved. Table 2 summarises the results.

Table 2: Synthesis results (from `synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	67
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	$746.966\,\mu m^2$
Flip-flop area	$16 \times 25.024\,\mu m^2 = 400.384\,\mu m^2$
Flip-flop area (% of total)	53.60 %
3 most-used cell types	`dfrtp_1` × 16, `nand2_1` × 12, `xnor2_1` × 7
Netlist simulation result	`PASS: 256 vectors checked, 0 errors` (Figure 4)

All 16 flip-flops were mapped to `dfrtp_1`, a positive-edge D flip-flop with an active-low asynchronous reset, which matches the RTL. The remaining 51 cells implement the AND array and the adders. Besides simple gates, ABC used complex cells such as AND-OR-invert (`a21oi`, `a32oi`), majority (`maj3`, the carry function of a full adder) and three-input XNOR (`xnor3`) gates.

To confirm that synthesis did not change the behaviour, the netlist was simulated with the same testbench, using the functional Verilog models of the SKY130 cells (`-DFUNCTIONAL`). Figure 4 shows the result: the netlist passed all 256 vectors, the same as the RTL.

```
I have no name!@f7c50cdf349d:/foss/designs$ cd /foss/designs/mult4
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
  tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:56: $finish called at 5140000 (1ps)
```

Figure 4: Gate-level simulation of the synthesized netlist: the testbench reports `PASS: 256 vectors checked, 0 errors`.

4 Static Timing Analysis (Part C)

Timing was analysed with OpenSTA on the synthesized netlist, using the typical-corner library (`tt_025C_1v80`) only. The script `sta/sta.tcl` defines the clock on `clk` with period T, sets an input delay of 1.0 ns on all inputs except the clock and an output delay of 1.0 ns on all outputs, and reports the worst setup slack and the critical path. At this stage there are no wires and the clock is ideal, so only cell delays are included.

At $T = 10$ ns, the worst slack was WNS = 7.396 ns, so the minimum clock period is

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.396 = 2.604\,\mathrm{ns}, \qquad F_{\max} = \frac{1000}{T_{\min}} = 384.0\,\mathrm{MHz}.$$

A second run at $T = 2.60$ ns ($T_{\min}$ rounded to 0.01 ns) gave a slack of -0.004 ns, confirming the result. The small negative value is due to rounding $T_{\min}$ down. The data arrival time on the critical path was 2.471 ns in both runs: changing the period moves only the required time, not the delays of the cells. The asynchronous reset (recovery) check was met with a large margin in both runs (9.228 ns and 1.828 ns), so it does not limit the clock frequency.

Table 3: Static timing analysis results (typical corner).

Metric	Value
Worst slack at $T = 10\,\text{ns}$	7.396 ns
$T_{\min}$	2.604 ns
$F_{\max}$	384.0 MHz
Slack at $T = T_{\min}$ (2.60 ns)	−0.004 ns
Critical path start point	`b_r[0]` (input register, flip-flop `_106_`)
Critical path end point	`p[6]` (output register, flip-flop `_100_`)
Number of cells on the critical path	8 logic cells (10 including both flip-flops)
Latency $= 2 \times T_{\min}$	5.21 ns
Throughput	384 million results per second

The critical path (Table 4) starts at `b_r[0]`, which drives all four AND gates of row 0, and ends at `p[6]`, the sum output of the last full adder `fa3_3`. It therefore crosses the whole array, from the first partial product to the last adder of the last row.

Table 4: Cells on the critical path, `b_r[0]` → `p[6]`, from `sta/mult_array_timing.rpt`.

#	Cell	Delay (ns)	Arrival (ns)	Function
—	`dfrtp_1` (`_106_`, `b_r[0]`)	0.388	0.388	clock-to-Q of launching flip-flop
1	`and2_0`	0.180	0.568	partial product
2	`or3_1`	0.391	0.959	adder logic
3	`a32oi_1`	0.250	1.208	adder logic
4	`o21ba_1`	0.198	1.406	adder logic
5	`xnor2_1`	0.147	1.553	sum (XOR)
6	`maj3_1`	0.385	1.937	carry
7	`maj3_1`	0.358	2.295	carry
8	`xnor3_1`	0.175	2.471	sum of `fa3_3` → `p[6]`
—	`dfrtp_1` (`_100_`, `p[6]`)	setup 0.134	—	capturing flip-flop

Of the 8 logic cells, one is the partial-product AND gate and **7 come from the full adders**, slightly fewer than the estimate of about $2N = 8$ full-adder delays. Section 6 explains the difference.

5 Physical Design (Part D)

The layout was produced with LibreLane, which runs synthesis, floorplanning, placement, clock tree synthesis, routing, parasitic extraction, signoff timing, DRC and LVS. The configuration `pnr/config.json` sets the design name, the two RTL files, the clock port `clk`, a fixed die of 150 µm × 150 µm, and the clock period. The initial clock period was $1.2 \times T_{\min} = 1.2 \times 2.604 = 3.125$ ns, rounded up to 3.5 ns.

Unlike Part C, LibreLane's signoff timing checks three process–voltage–temperature (PVT) corners: typical (`tt_025C_1v80`), fast (`ff_n40C_1v95`) and slow (`ss_100C_1v60`), each with minimum, nominal and maximum wire parasitics. At 3.5 ns, the typical and fast corners passed, but the slow corner failed by 2.118 ns, because cells there are roughly twice as slow. Following the handout, `CLOCK_PERIOD` was increased by 0.5 ns per run until all corners met setup; every run is recorded in the README. DRC and LVS were clean in every run.

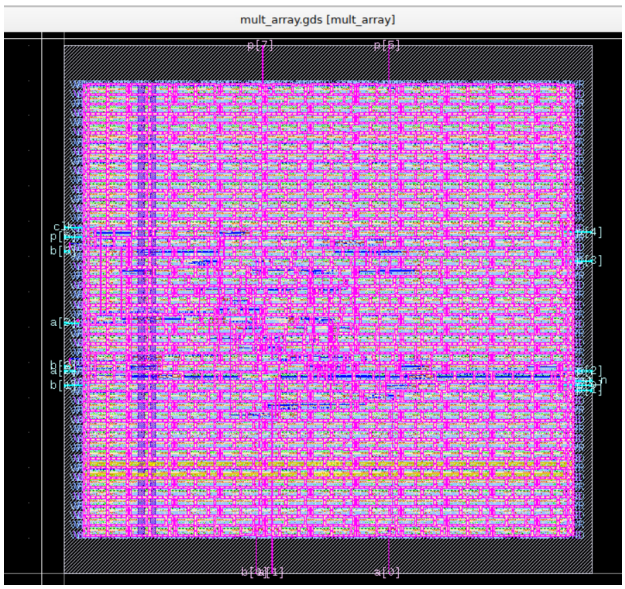
In runs 1–5, the slow corner consistently needed about 5.62 ns. In run 6, the slack became *worse* despite the longer period: with more time available, the timing optimiser stopped resizing cells earlier, and after routing the slightly slower circuit failed by a small margin. Run 8 at 7.0 ns passed in all corners, with 0 DRC and 0 LVS errors. Table 5 gives the final results.

Table 5: Physical design results (final run, `CLOCK_PERIOD` = 7.0 ns).

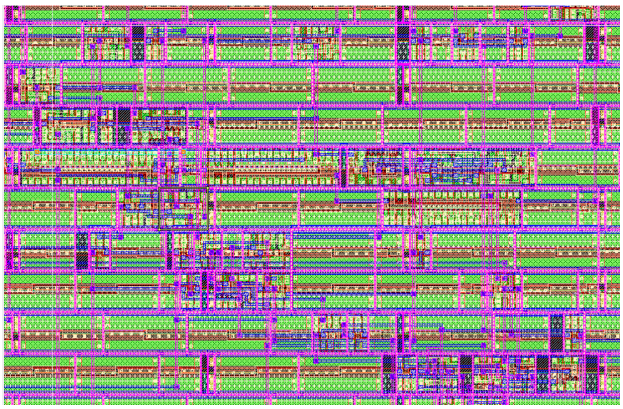
Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	8
Core area	17 759.5 µm^2 (die: 22 500 µm^2)
Cells after synthesis (Part B)	67
Cells after place and route	1883 (328 excluding fill and decap cells)
Kinds of cells added	1299 decap, 256 fill, 245 tap, 19 timing-repair buffers (`buf_2`, `clkdlybuf4s50_1`), 3 clock buffers (`clkbuf_16`)
Wirelength	2333 µm
Setup slack	+0.465 ns (worst corner, `max_ss_100C_1v60`)
Post-route $F_{\max}$	153.0 MHz (worst corner); 316.4 MHz (typical corner)
$F_{\max}$ from Part C	384.0 MHz (typical corner)
DRC errors	0 (Magic and KLayout)
LVS errors	0

Post-route $F_{\max}$ was calculated as 1000/(`CLOCK_PERIOD` − setup slack). Using the overall worst slack gives $1000/(7.0 - 0.465) = 153.0$ MHz. For a fair comparison with Part C, which used only the typical corner, the typical-corner slack of 3.839 ns gives $1000/(7.0 - 3.839) = 316.4$ MHz, about 18 % below the 384.0 MHz from Part C.

The number of cells grew from 67 after synthesis to 1883 after place and route. LibreLane re-synthesizes the RTL with its own settings and produced 61 logic cells (16 flip-flops and 45 gates) instead of 67; all the remaining cells were added by the physical design flow. The kinds of added cells were identified by counting the cell names in the final DEF file (`final/def/mult_array.def`); Section 6 explains why each kind is needed. Figure 5 shows the final layout.



(a) Whole 150 µm × 150 µm die, with I/O pins on the edges and the power grid.



(b) Zoomed view showing standard-cell rows and routed metal wires.

Figure 5: Final layout of `mult_array` in KLayout.

6 Answers to Questions

Q1. ***The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply 16 + 12.***

Synthesis produced 67 cells, of which 16 are flip-flops, so **51 cells** implement the logic, compared with $16 + 12 = 28$ if every AND gate and full adder were one cell. The count differs because the full adders and AND gates do not map one-to-one onto library cells. SKY130 does have a full-adder cell (`fa_1`), but it has two outputs, and ABC maps logic only onto single-output cells (Yosys reported "Detected 9 multi-output cells", for example `sky130_fd_sc_hd__fa_1`). Each full adder is therefore built from several gates, such as an XNOR for the sum and a majority gate (`maj3`) for the carry. In the other direction, flattening let ABC optimise across adder boundaries: most of the 16 AND gates were merged into the adder logic (only 2 `and2` cells remain, alongside 12 `nand2`), and the row-1 adders with constant-0 inputs ($r_0[3] = 0$, $c_1[0] = 0$) were simplified. Built from simple gates (2 XOR + 2 AND + 1 OR per adder), the array would need $16 + 12 \times 5 = 76$ cells, so 51 is a substantial saving.

Q2. ***What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?***

The critical path runs from the input register `b_r[0]` to the output register `p[6]`, the sum output of the last full adder `fa3_3`. `b_r[0]` drives all four row-0 AND gates, so the path starts at the top of the array and ends at its far corner. It contains **8 logic cells** (10 including the two flip-flops; Table 4): one AND gate and 7 cells doing adder work, compared with the estimate of about $2N = 8$ full-adder delays. The count is slightly lower because ABC merged logic across adder boundaries, so cells such as `or3`, `a32oi` and `o21ba` each perform part of more than one adder, and because the row-1 adders with constant-0 inputs were simplified. The structure still matches the estimate: the path alternates between carry cells (`maj3`) and sum cells (`xnor2`, `xnor3`) as it crosses the rows.

Q3. ***What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?***

Flip-flops take 400.384 µm^2 of 746.966 µm^2, i.e. 53.60 % of the cell area, although they are only 16 of 67 cells. One `dfrtp_1` flip-flop has an area of 25.02 µm^2, about 6.7 times a `nand2_1` (3.75 µm^2), because it contains two latches plus asynchronous-reset circuitry, while the 51 logic cells average only 6.8 µm^2. The share is large because the design is small: the logic grows roughly with N^2 (N^2 AND gates and $N(N-1)$ full adders), whereas the number of flip-flops grows only with $4N$ ($2N$ input and $2N$ output bits). At $N = 4$, the 16 flip-flops therefore outweigh the logic; for a wider multiplier, the logic would dominate.

Q4. ***Why is the post-route $F_{\max}$ lower than the $F_{\max}$ from Part C? Which kinds of cells did the flow add, and why is each kind needed?***

Part C gave 384.0 MHz under ideal conditions: no wires, an ideal clock, and the typical corner only. After place and route, timing includes the resistance and capacitance of 2333 µm of routed wire and the delay and skew of the real clock tree. Even in the typical corner, $F_{\max}$ drops to 316.4 MHz (18 % lower). The handout's formula uses the worst slack over all corners, which comes from the slow corner `ss_100C_1v60`, and gives 153.0 MHz. The flow added five kinds of cells. **Timing-repair buffers** (19) split long or heavily loaded nets so that signals switch faster. **Clock buffers** (3 `clkbuf_16`) form the clock tree, delivering `clk` to all 16 flip-flops at nearly the same time. **Tap cells** (245) connect the n-wells to VDD and the substrate to ground at regular intervals to prevent latch-up. **Decap cells** (1299) are capacitors between VDD and ground that keep the supply stable when many gates switch at once. **Fill cells** (256) fill the remaining gaps, so the wells, implants and power rails are continuous along every row, as manufacturing requires.

Q5. ***What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?***

DRC (design rule check) verifies that every shape in the layout obeys the foundry's manufacturing rules, such as minimum width, minimum spacing, enclosure of contacts and vias, and layer density. **LVS** (layout versus schematic) extracts the transistors and their connections from the layout and checks that they match the netlist, i.e. that the drawn layout is the intended circuit. Both are needed because each catches errors the other cannot: a DRC violation can cause shorts, opens or low yield even if the circuit is correct, while a layout that is DRC-clean can still be wired wrongly and compute

the wrong function. The final layout has **0 DRC errors** (Magic and KLayout) and **0 LVS errors** (Netgen).

Q6. ***Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.***

Using the formulas from Section 2 of the assignment handout, N^2 AND gates and $N(N-1)$ full adders (Table 6), the hardware grows roughly with N^2, but the delay only about linearly, doubling from $N = 4$ to $N = 8$.

Table 6: Estimated scaling from a 4-bit to an 8-bit array multiplier.

	$N = 4$	$N = 8$	Change
AND gates (N^2)	16	64	$\times 4$
Full adders ($N(N-1)$)	12	$8 \times 7 = 56$	$\times 4.7$
Critical path ($\approx 2N$ FA delays)	8	16	$\times 2$

Scaling the measured logic delay of the critical path, $2.471 - 0.388 = 2.083$ ns, by 2 gives about 4.17 ns. Adding the flip-flop clock-to-Q delay (0.388 ns) and setup time (0.134 ns) gives

$$T_{\min} \approx 4.17 + 0.52 = 4.69\,\text{ns}, \qquad F_{\max} \approx \frac{1000}{4.69} \approx 213\,\text{MHz}$$

in the typical corner before layout. This is only an estimate, since ABC would optimise the larger array differently.

7 Conclusion

A 4-bit unsigned array multiplier was designed in Verilog and taken from RTL to a manufacturable GDSII layout on the SkyWater SKY130 process using only open-source tools. An exhaustive, self-checking testbench verified all 256 input combinations on both the RTL and the synthesized netlist, and an error test confirmed that it detects faults. Synthesis produced 67 cells with an area of 746.966 µm^2, more than half of which is flip-flops. Static timing analysis found a critical path from `b_r[0]` to `p[6]` and a maximum frequency of 384.0 MHz in the typical corner. After place and route, the design met timing in all PVT corners at a clock period of 7.0 ns, with 0 DRC and 0 LVS errors.

The main lessons were that the structure of a synthesized circuit differs considerably from the RTL, since the tools merge and restructure logic; that timing estimated before layout is optimistic, because wires, the clock tree and slow corners reduce the achievable frequency; and that most of the cells in the final layout are not logic but tap, fill and decap cells needed for manufacturing and power integrity.

Repository. All source files, scripts, generated reports and the complete list of commands are available at `https://github.com/mrityunjay-84/mult4`.

Team contributions. Mrityunjay Badoni (2026203019) and Shreekara Aithala (2026203001) carried out Parts A to D together, from RTL design and verification through synthesis and static timing analysis to place and route. Shreekara Aithala prepared the figures, including the block diagram, the hand-worked example and the screenshots, and Mrityunjay Badoni wrote the report.

Use of AI tools. An AI assistant (Claude) was used for learning the concepts and tools and for debugging. All design work, simulations and tool runs were carried out by the team, and all results were checked against the tool outputs.